

Step 2 — Buying a Policy: `subscribeToPolicy` and Premium Payment (EIP-20)

This is the first step that actually touches the blockchain. Alice picks a plan and pays for it — atomically, in one transaction, per the fix built earlier in this project specifically to prevent a policy existing without payment ever completing.

EIP-20 in this step

Every value transfer here — Alice's approval, the actual payment pull — is a plain, standard ERC-20 (EIP-20) interaction. No custom token logic is involved in the *movement* of funds; `StableCoin.sol` only adds custom logic for privileged `mint / burn` (used elsewhere, see Step 6), never for ordinary transfers.

The actual sequence – Alice buying the Premium plan (\$33)

1. StableCoin.approve — Alice authorizes InsurancePolicy to spend her dUSD

- **Contract:** `StableCoin` at `0x55555555555555555555555555555555EEEE`
- **Function:** `approve(address spender, uint256 amount)` — inherited directly from `OpenZeppelin's ERC20`, not overridden by `StableCoin.sol`
- **Input:** `spender = 0x222222222222222222222222222222228BBB` (`InsurancePolicy`'s proxy address), `amount = 33 ether`
- **Traversal:** writes `_allowances[0x1111...aaaa][0x2222...bbbb] = 33000000000000000000` directly into `StableCoin`'s storage — no other logic
- **Output:** `bool ( true );` emits the standard EIP-20 event `Approval(0x1111...aaaa, 0x2222...bbbb, 33 ether)`
- **Imported contracts:** none beyond `OpenZeppelin's` own `ERC20.sol` internals

2. InsurancePolicy.subscribeToPolicy – the actual purchase

- **Contract:** InsurancePolicy at 0x222222222222222222222222222222B8BB
- **Function:** subscribeToPolicy(uint256 templateId)
- **Input:** templateId = 3 (the Premium plan)
- **Imported contracts used by this function:** none directly (reads its own policyTemplates storage; the actual token movement happens in the internal _collectPremium call, step 3 below, which imports IERC20 / SafeERC20)
- **Traversal, value by value:**
 1. Reads policyTemplates[3] from storage → PolicyTemplate{ coverageAmount: 10_000 ether, premiumAmountPerPeriod: 33 ether, periodSeconds: 2_592_000 (30 days in seconds), termSeconds: 31_536_000 (365 days), active: true, metadataHash: 0xabc...}.
 2. Checks premiumAmountPerPeriod != 0 (template exists) and active == true — both pass.
 3. Reads nextPolicyId from storage — say it's currently 1 (Alice is the first ever subscriber in this trace). Checks _exists[1] == false (true, so no collision-skip needed). policyId is assigned 1; nextPolicyId is written back as 2.
 4. Builds and stores into policies[1]:

```
Policy {
  policyId: 1,
```

```

    coverageAmount: 10_000 ether,
    holder: 0x11111111111111111111111111111111AAAA,
    validFrom: 1_800_000_000,           // block.timestamp at this tx
    validUntil: 1_831_536_000,         // validFrom + 365 days
    revoked: false,
    metadataHash: 0xabc...
}

```

```
_exists[1] = true.
```

- Stores into `premiumTerms[1]: PremiumTerms{ amountPerPeriod: 33 ether, periodSeconds: 2_592_000 }` — copied from the template, locking in Alice's rate even if the template's price changes later.
- Emits `PolicyRegistered(1, 0x1111...aaaa, 1_800_000_000, 1_831_536_000, 10_000 ether, 0xabc...)` and `PolicySubscribed(1, 3, 0x1111...aaaa, 1_831_536_000)`.
- Calls the internal `_collectPremium(1, 0x1111...aaaa)` — **still inside this same transaction; if this reverts, everything above reverts too, including the Policy record just written.**

- **Output:** `uint256 policyId = 1`, returned to whatever called `subscribeToPolicy` (the frontend, via the wallet's transaction).

3. `_collectPremium` (private, called only from step 2.7) — the payment itself

- **Function:** `_collectPremium(uint256 policyId, address payer)`
- **Input:** `policyId = 1`, `payer = 0x1111...aaaa`
- **Imported contracts used:** `IERC20`, `SafeERC20` — `SafeERC20.safeTransferFrom` is what actually calls into `StableCoin` next
- **Traversal:**
 - Checks `premiumToken != address(0)` — `0x5555...eeee`, set.
 - Reads `premiumTerms[1] → { amountPerPeriod: 33 ether, periodSeconds: 2_592_000 }`.
 - `base = max(premiumPaidUntil[1], block.timestamp)`. Since this policy is brand new, `premiumPaidUntil[1]` is `0` (default), so `base = block.timestamp = 1_800_000_000`.
 - `newPaidUntil = 1_800_000_000 + 2_592_000 = 1_802_592_000`. Written into `premiumPaidUntil[1]`.
 - Calls `premiumToken.safeTransferFrom(0x1111...aaaa, 0x6666...ffff, 33 ether)` — an external call into `StableCoin` (step 4 below).
 - Emits `PremiumPaid(1, 0x1111...aaaa, 33 ether, 1_802_592_000)`.

4. `StableCoin.transferFrom` — the actual dUSD movement (EIP-20)

- **Contract:** `StableCoin` at `0x55555555555555555555555555555555EEEE`
- **Function:** `transferFrom(address from, address to, uint256 amount)` — again, plain inherited `ERC20`, not custom
- **Input:** `from = 0x1111...aaaa`, `to = 0x6666...ffff` (Premium Treasury), `amount = 33 ether`
- **Traversal:** checks `_allowances[0x1111...aaaa][0x2222...bbbb] >= 33 ether` (set in step 1), decrements that allowance to `0`, decrements `_balances[0x1111...aaaa]` by `33 ether`, increments `_balances[0x6666...ffff]` by `33 ether`.
- **Output:** `bool (true)`; emits `Transfer(0x1111...aaaa, 0x6666...ffff, 33 ether)`.

Value trace summary — where "33" actually goes

```

Alice's wallet balance:      -33 ether  (33000000000000000000 wei-units)
Premium Treasury balance:    +33 ether
Alice's dUSD allowance to InsurancePolicy: 33 ether → 0 (fully consumed)
premiumPaidUntil[1]:        0 → 1_802_592_000 (30 days from now)
_policies[1].holder:         0x1111...aaaa (Alice, permanently recorded)

```

If step 4 had failed (e.g. Alice's allowance was too low), **every write in step 2 rolls back too** — there is no reachable state where `_policies[1]` exists but the payment didn't happen. This atomicity is the specific fix built earlier in this project for exactly this failure mode.